Third Year Engineering

23CSE3006 -MACHINE LEARNING

Class - T.Y. AIA (SEM-II)

Unit III: Unsupervised Machine Learning

Name Of the Course Coordinator:

Prof. Aarti Pimpalkar

Team Members

1. Prof. Dr. Nilima Kulkarni
2. Prof. Dr. Reena Gunjan
3. Prof. Dattatray Kale
4. Prof. Nilesh Kulal
5. Prof. Hemant Shinde
6. Prof. Ashwini shahapurkar
7. Prof. Roshni Bhave

AY 2026-2027 SEM-I

Syllabus :UNIT III

9 hours

- Unsupervised Machine Learning Applications;
- Clustering Tasks;
- Different types of Clustering techniques;
- K-means clustering;
- K-medoids;
- Agglomerative hierarchical clustering,
- Evaluating performance of clustering models;

Introduction to Unsupervised Learning

Definition

- **Unsupervised Learning** is a type of machine learning technique where the model is not provided with labeled training data.
- The algorithm explores the data and tries to identify patterns, structures, or relationships **without any explicit guidance or feedback**.
- It contrasts with **supervised learning**, where the input comes with labeled output.

Goal

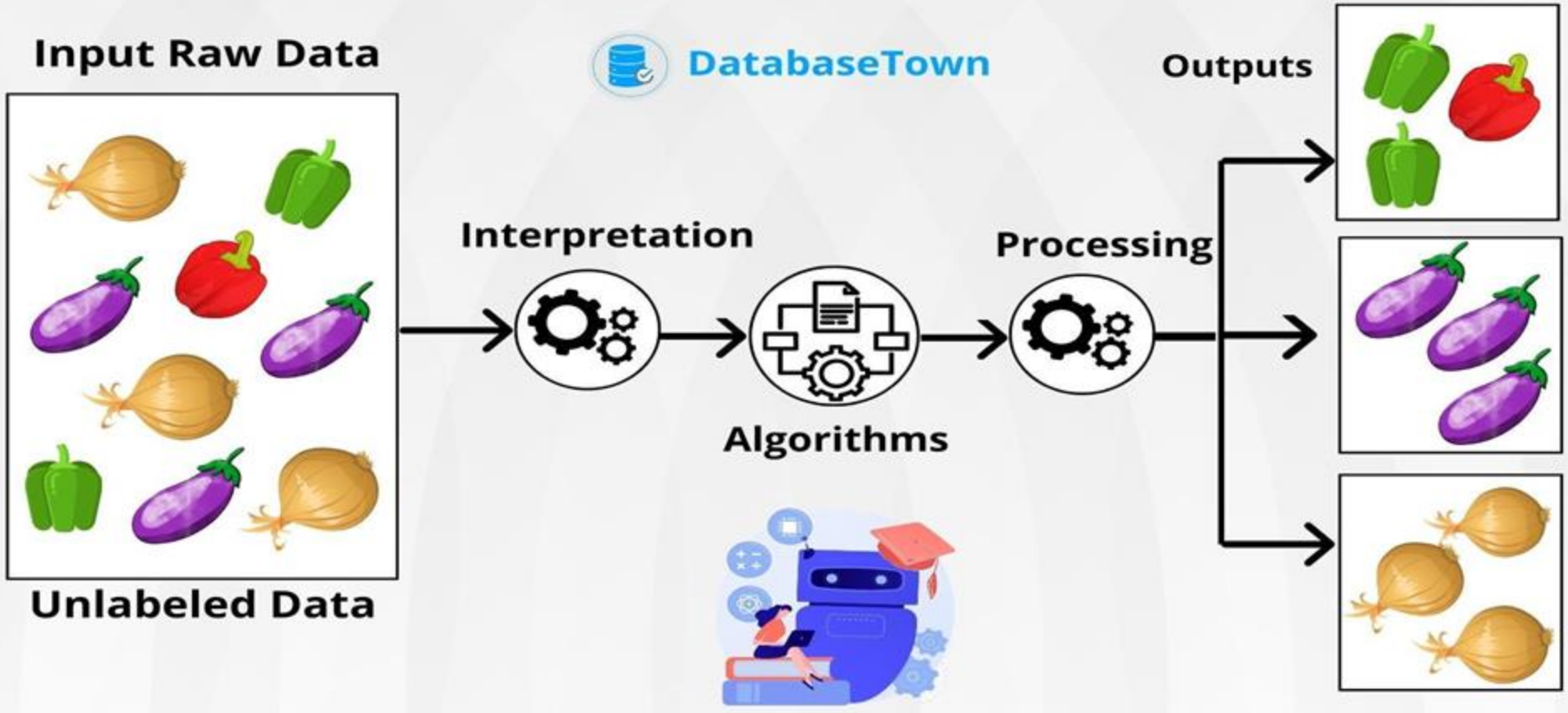
- The primary goal is to **uncover hidden patterns** in datasets.
- It attempts to learn the **underlying structure or distribution** in the data.
- It helps in understanding data better or preparing it for further supervised learning processes.

supervised Learning Vs Unsupervised Learning



UNSUPERVISED LEARNING

Unsupervised learning is a type of machine learning where the algorithm learns from unlabeled data without any predefined outputs or target variables.



Common Tasks in Unsupervised Learning

1. Clustering:
Grouping similar data points into clusters.
Example: Segmenting customers into different groups based on buying behavior.
2. Dimensionality Reduction:
Reducing the number of features while retaining essential data characteristics.
Example: Principal Component Analysis (PCA), t-SNE.
3. Anomaly Detection:
Identifying rare data points that deviate significantly from the rest.
Example: Fraud detection in transactions.

What is Clustering?

Definition of Clustering

- **Clustering** is an **unsupervised machine learning technique** used to group a set of **data points into clusters** such that:
- Data points within the same cluster are **more similar** to each other.
- Data points from different clusters are **dissimilar**.

It helps uncover **natural structures** in data without the use of pre-labeled outputs.

Purpose of Clustering

- Discover hidden **patterns** or **group behaviors** in datasets.
- Reduce data complexity by summarizing large datasets into smaller, meaningful groups.
- Enhance data analysis, visualization, and decision-making processes.

Benefits of Clustering

No need for labeled data

Enables **data exploration** and hypothesis generation

Enhances understanding of **data structure and distribution**

Example of Clustering : E-Commerce Use Case

An online shopping site tracks user behavior:

Time spent on site, purchase frequency, items viewed.

Clustering groups users into categories like:

High spenders

Occasional browsers

Deal hunters

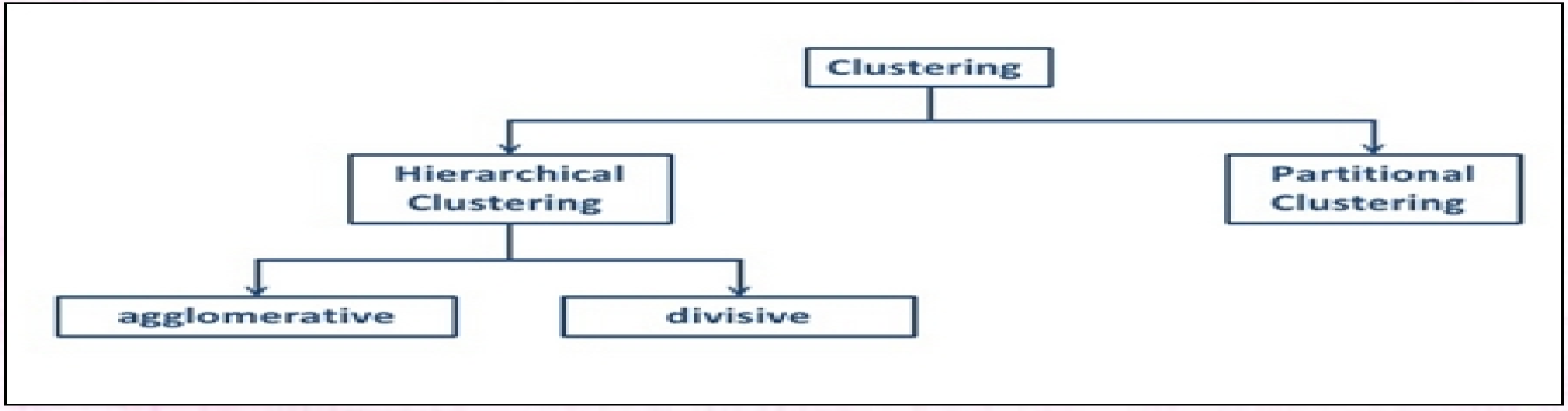
This enables **targeted marketing** and **personalized recommendations**.



Types of clustering

Cluster analysis is all about the kind of algorithms that can be used to find clusters automatically given the data. There are primarily two classes of clustering algorithm; they are as follows:

- The Hierarchical Clustering algorithms
- The Partitional Clustering algorithms



1. Partitioning Clustering

Divides the data into **k non-overlapping subsets (clusters)**.

Each data point belongs to **only one cluster**.

Tries to **optimize intra-cluster similarity** and **minimize inter-cluster similarity**.

Algorithms:

K-means: Uses centroid-based distance.

K-medoids: Uses actual data points as cluster centers (medoids).

2. Hierarchical Clustering

Builds a **tree-like structure (dendrogram)** representing nested clusters.

Two types:

Agglomerative (Bottom-up): Start with individual points, merge step-by-step.

Divisive (Top-down): Start with one cluster, recursively split.

Linkage Methods:

Single linkage (nearest neighbor)

Complete linkage (farthest neighbor)

Average linkage

3. Density-Based Clustering

Clusters are formed based on **regions of high density** separated by regions of low density.

Good for **arbitrary-shaped clusters**.

Algorithm:

DBSCAN (Density-Based Spatial Clustering of Applications with Noise).

4. Grid-Based Clustering *(Less commonly covered but useful)*

The data space is divided into a **finite number of cells** (grids), and clustering is performed on these grids.

Algorithm: STING (Statistical Information Grid), CLIQUE.

5. Model-Based Clustering

Assumes data is generated from a mixture of **underlying probability distributions** (e.g., Gaussian).

Algorithm: Gaussian Mixture Model (GMM) using **Expectation-Maximization (EM)**.

Introduction to K-Means Clustering

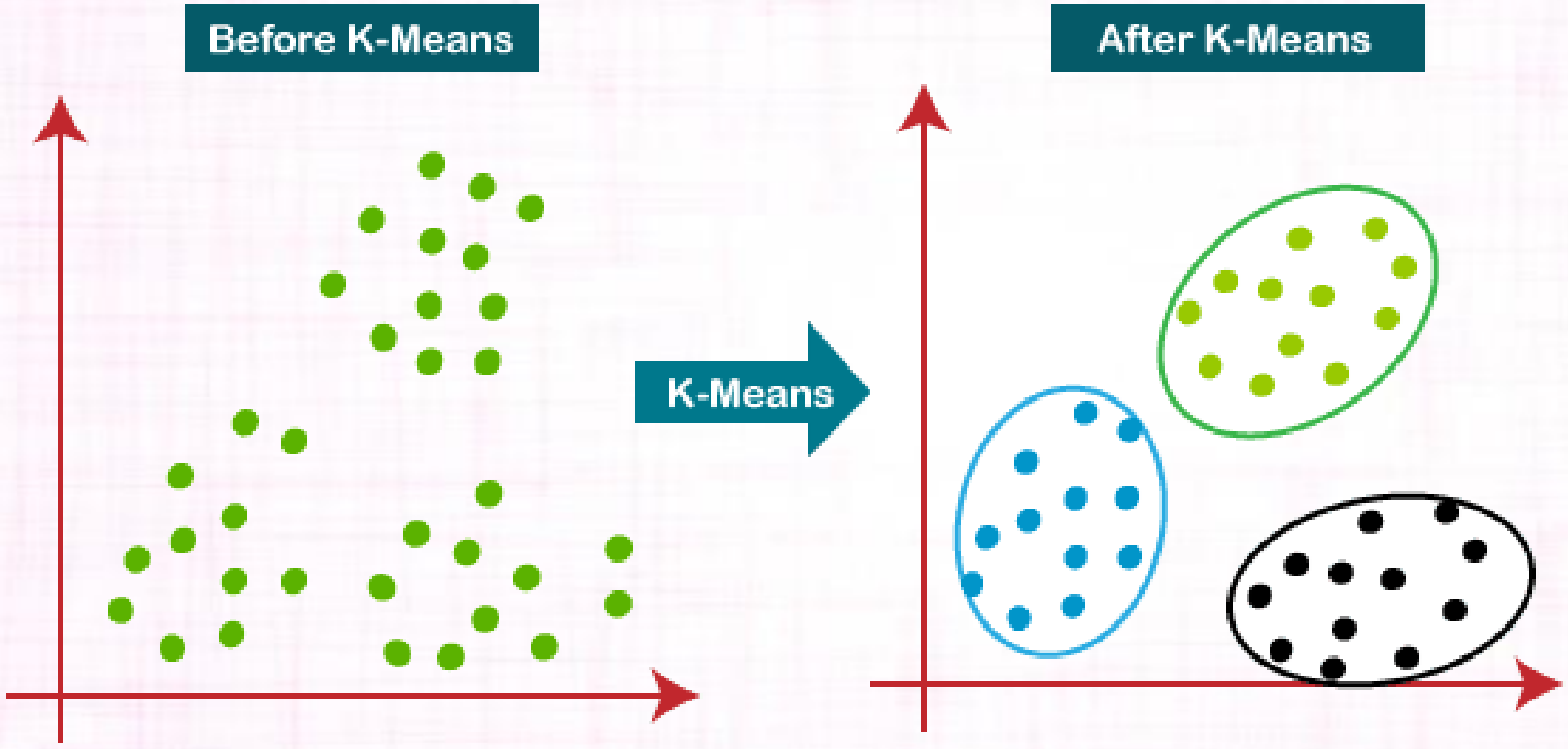
In the field of machine learning, one of the key objectives is to achieve accurate predictions. These predictions can be achieved through various learning algorithms, categorized as supervised and unsupervised.

K-means clustering is one such unsupervised algorithm that groups data points based on their similarity. In this blog, we will explore the use cases, advantages, and working principles of the K-means clustering algorithm.

What is K-means Clustering?

K-means clustering is an unsupervised machine learning approach that uses similarities to divide a dataset into K different clusters.

The algorithm aims to minimize the within-cluster variance, ensuring that data points within the same cluster are as similar as possible while data points in different clusters are as dissimilar as possible.


Before K-Means
K-Means
After K-Means

Why K-Means Clustering Algorithm is Widely Popular?

The K-means clustering algorithm has gained remarkable popularity in the field of machine learning and data analysis due to several compelling reasons. Its simplicity, versatility, and effectiveness in solving a wide array of problems have positioned it as a staple tool in the data scientist's toolkit.

1. Simplicity and Ease of Implementation
2. Speed and Efficiency
3. Unsupervised Learning Capability
4. Versatility Across Domains
5. Interpretable Results
6. Scalability and Resource Efficiency

How K-Means work

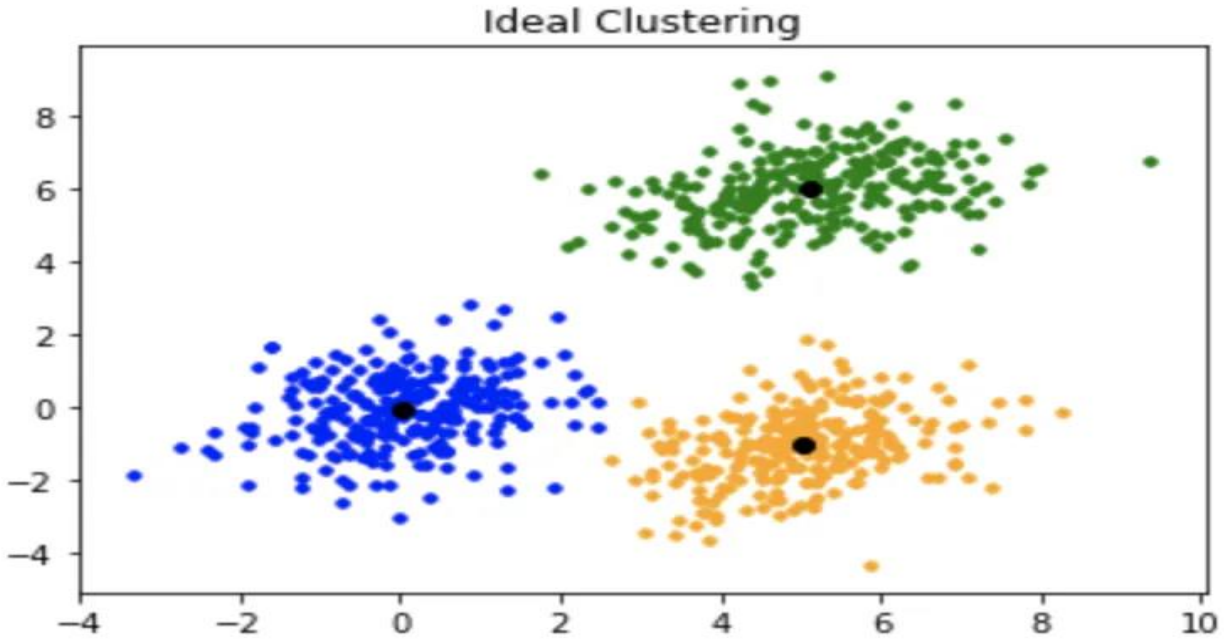
1. **Initialization:** Randomly select K data points as cluster center
2. **Calculate Distance:** Calculate Distance between each data point and each cluster centre.

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. **Assignment**: Assign each data point to that cluster whose center is nearest to that data point
4. **Update centroids(Centres):** Re-compute the center of newly formed clusters
   - The center of a cluster is computed by taking the mean of all the data points contained in that cluster

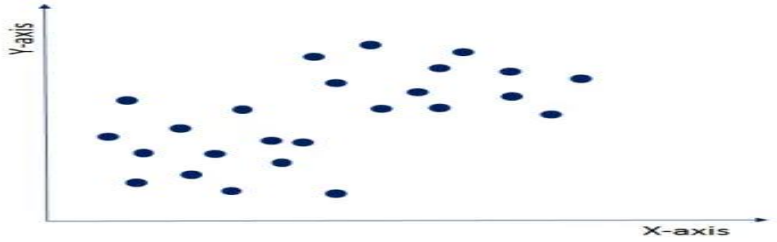
5. Repeat: Repeat steps 2 and 3 until any of the following stopping criteria is met-

- If data points fall in the same cluster
- Reached maximum of iteration
- The newly formed cluster does not change in center points



Determining Number of Clusters

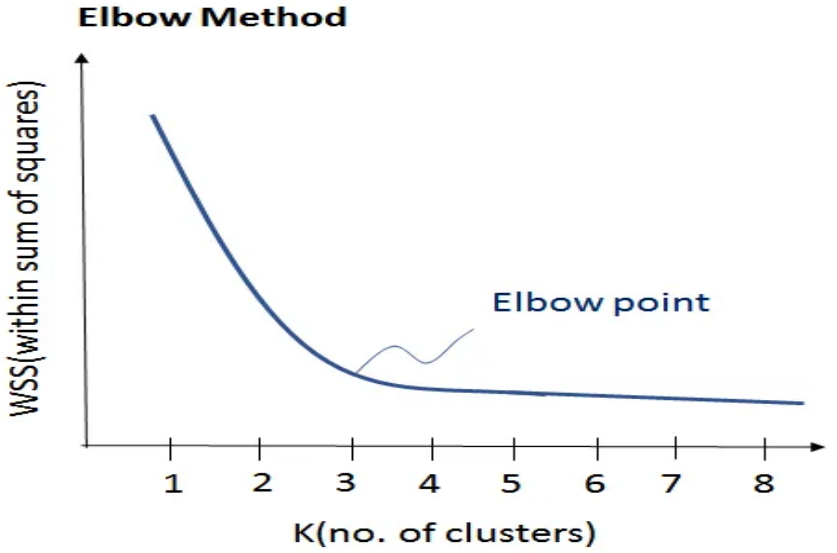
- Let’s say you have an unlabeled data set like the one shown below and you want to group this data into clusters.



- how should you choose the optimum number of clusters?
  - **Elbow Method:** technique that we use to determine the number of centroids(k) to use in a k-means
- We draw a curve between WCSS (within cluster sum of squares) and the number of clusters
- curve looks like a human arm and the elbow point gives us the optimum number of clusters

Elbow Method

- determine the k-value we continuously iterate for k=1 to k=n
- For every value of k, we calculate the within-cluster sum of squares (WCSS) value
- For determining the best number of clusters(k) we plot a graph of k versus their WCSS value
- When k=1 the WCSS has the highest value
- with increasing k value WCSS value decreases
- We select that value of k where line is straight



Solved Example of K- Means Clustering

Problem Statement

We have marks of **12 students in Maths**:

10, 12, 11, 25, 28, 30, 27, 50, 52, 48, 60, 62

We want to group them into **3 clusters (K=3)** using **Euclidean distance**.

Euclidean distance in 1D:

$$d(x, y) = \sqrt{(x - y)^2} = |x - y|$$

Step 1: Initialization

Let's choose **3 initial centroids** (pick first, middle, and last values for simplicity):

Centroid1 = 10

Centroid2 = 25

Centroid3 = 50

Step 2 — Compute distances and assign each point to nearest centroid

Below each point's distances to centroids (rounded to 1 decimal where necessary) and the chosen cluster:

Point **10**

Below each point's distances to centroids (rounded to 1 decimal where necessary) and the chosen cluster:

Point **10**

d(10,10) = |10 − 10| = 0.0

d(10,25) = |10 − 25| = 15.0

d(10,50) = |10 − 50| = 40.0

→ nearest = Centroid1 → **Cluster 1**

Point **12**

d(12,10) = |12 − 10| = 2.0

d(12,25) = |12 − 25| = 13.0

d(12,50) = |12 − 50| = 38.0

→ nearest = Centroid1 → **Cluster 1**

Point 11
d(11,10) = |11 − 10| = 1.0
d(11,25) = |11 − 25| = 14.0
d(11,50) = |11 − 50| = 39.0
→ nearest = Centroid1 → **Cluster 1**
Point 25
d(25,10) = |25 − 10| = 15.0
d(25,25) = |25 − 25| = 0.0
d(25,50) = |25 − 50| = 25.0
→ nearest = Centroid2 → **Cluster 2**
Point 28
d(28,10) = 18.0
d(28,25) = 3.0
d(28,50) = 22.0
→ nearest = Centroid2 → **Cluster 2**

Point 30
d(30,10) = 20.0
d(30,25) = 5.0
d(30,50) = 20.0
→ nearest = Centroid2 (tie between centroids 1 and 3 not possible here; nearest is Centroid2 with 5.0) → **Cluster 2**
Point 27
d(27,10) = 17.0
d(27,25) = 2.0
d(27,50) = 23.0
→ nearest = Centroid2 → **Cluster 2**
Point 50
d(50,10) = 40.0
d(50,25) = 25.0
d(50,50) = 0.0
→ nearest = Centroid3 → **Cluster 3**

Point 52
d(52,10) = 42.0
d(52,25) = 27.0
d(52,50) = 2.0
→ nearest = Centroid3 → **Cluster 3**
Point 48
d(48,10) = 38.0
d(48,25) = 23.0
d(48,50) = 2.0
→ nearest = Centroid3 → **Cluster 3**
Point 60
d(60,10) = 50.0
d(60,25) = 35.0
d(60,50) = 10.0
→ nearest = Centroid3 → **Cluster 3**

Point 62
d(62,10) = 52.0
d(62,25) = 37.0
d(62,50) = 12.0
→ nearest = Centroid3 → **Cluster 3**
Clusters after first assignment:
Cluster 1: {10, 12, 11}
Cluster 2: {25, 28, 30, 27}
Cluster 3: {50, 52, 48, 60, 62}

Student	Value	d(x,10)	d(x,25)	d(x,50)	Cluster
A	10	0	15	40	C1
B	12	2	13	38	C1
C	11	1	14	39	C1
D	25	15	0	25	C2
E	28	18	3	22	C2
F	30	20	5	20	C2
G	27	17	2	23	C2
H	50	40	25	0	C3
I	52	42	27	2	C3
J	48	38	23	2	C3
K	60	50	35	10	C3
L	62	52	37	12	C3

Step 3 — Update centroids (mean of cluster members)

Compute average (mean) for each cluster:

New Centroid1 = (10 + 12 + 11) / 3 = 33 / 3 = **11.0**

New Centroid2 = (25 + 28 + 30 + 27) / 4 = 110 / 4 = **27.5**

New Centroid3 = (50 + 52 + 48 + 60 + 62) / 5 = 272 / 5 = **54.4**

So new centroids: **[11.0, 27.5, 54.4]**

Step 4 — Reassign using new centroids

Compute distances again to the new centroids and check assignments (showing a few examples):

For $_{10}$: d(10,11.0)=1.0, d(10,27.5)=17.5, d(10,54.4)=44.4 → Cluster 1

For $_{28}$: d(28,11.0)=17.0, d(28,27.5)=0.5, d(28,54.4)=26.4 → Cluster 2

For $_{50}$: d(50,11.0)=39.0, d(50,27.5)=22.5, d(50,54.4)=4.4 → Cluster 3

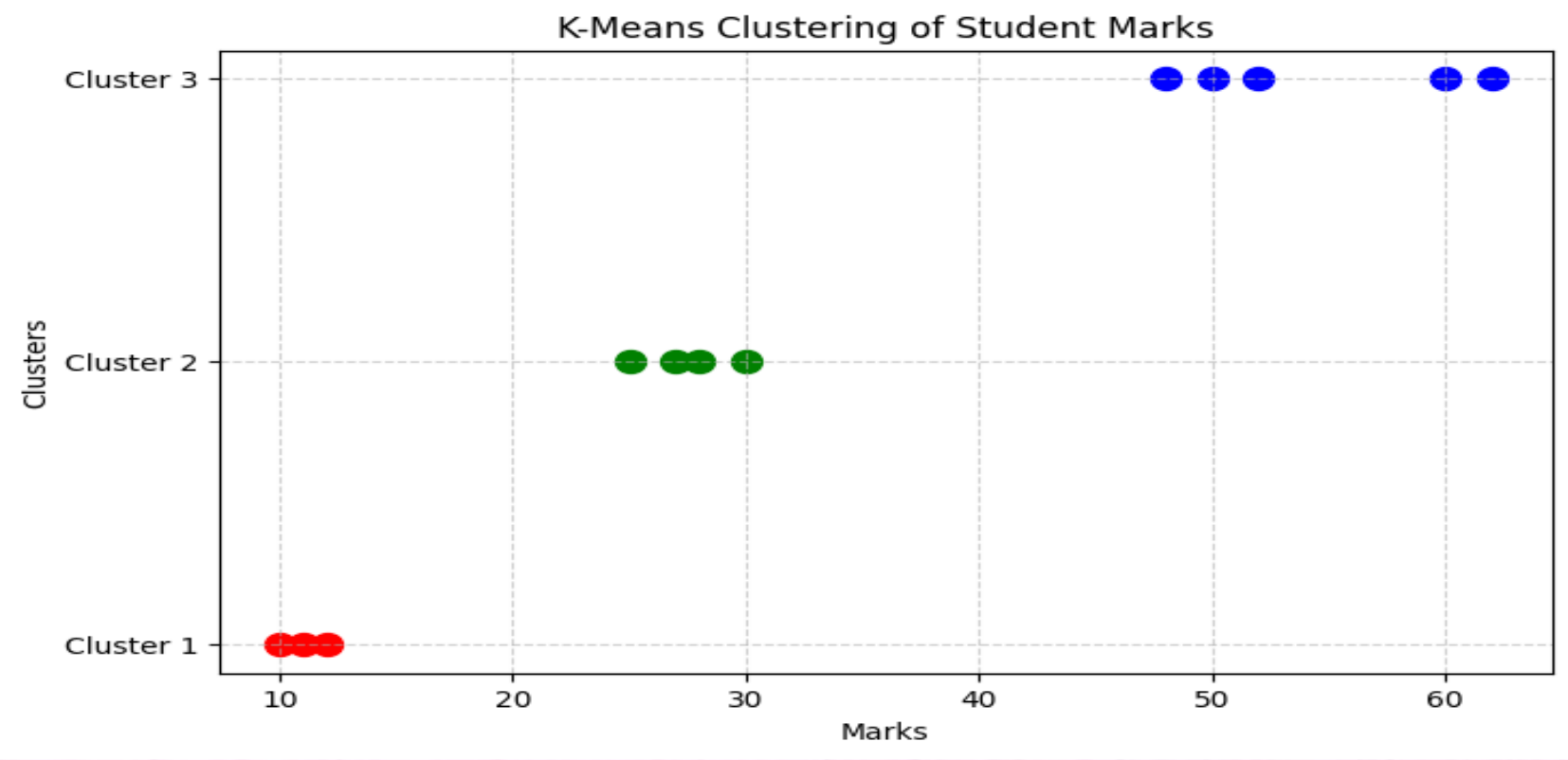
Final Result

Cluster 1 (Low marks): {10, 12, 11} → **Centroid = 11.0**

Cluster 2 (Medium marks): {25, 28, 30, 27} → **Centroid = 27.5**

Cluster 3 (High marks): {50, 52, 48, 60, 62} → **Centroid = 54.4**

The following graph shows these three clustors



K-Medoids clustering

K-Medoids (also called Partitioning Around Medoid) algorithm was proposed in 1987 by Kaufman and Rousseeuw. A medoid can be defined as a point in the cluster, whose dissimilarities with all the other points in the cluster are minimum.

Algorithm:

Initialize: select *k* random points out of the *n* data points as the medoids.

Associate each data point to the closest medoid by using any common distance metric methods.

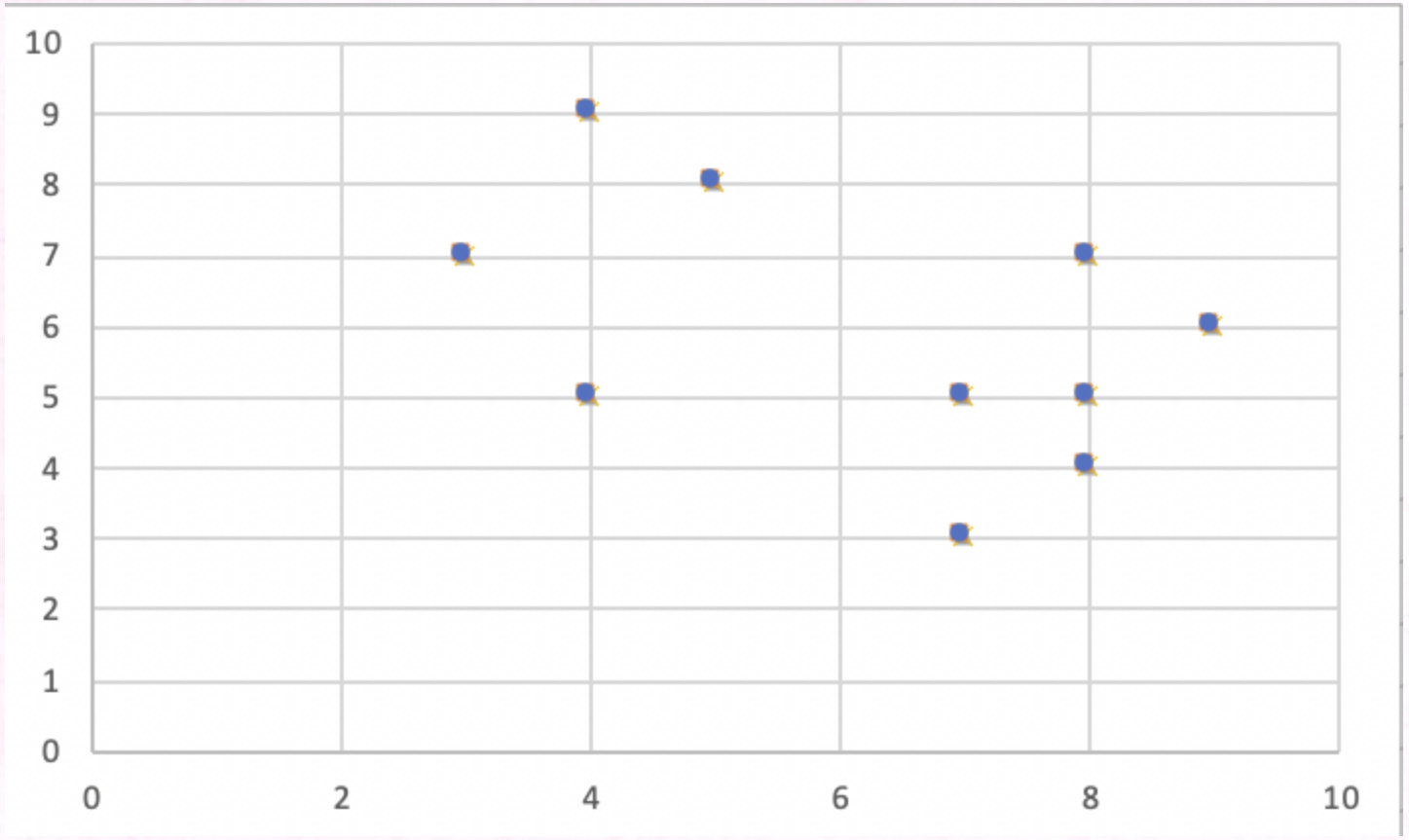
While the cost decreases: For each medoid m, for each data o point which is not a medoid:

Swap m and o, associate each data point to the closest medoid, and recompute the cost.

If the total cost is more than that in the previous step, undo the swap.

	X	Y
0	8	7
1	3	7
2	4	9
3	9	6
4	**8**	**5**
5	5	8
6	7	3
7	8	4
8	7	5
9	**4**	**5**

Let's consider the following example: If a graph is drawn using the above data points, we obtain the following:



Step 1: Let the randomly selected 2 medoids, so select k = 2, and let **C1 -(4, 5)** and **C2 -(8, 5)** are the two medoids.

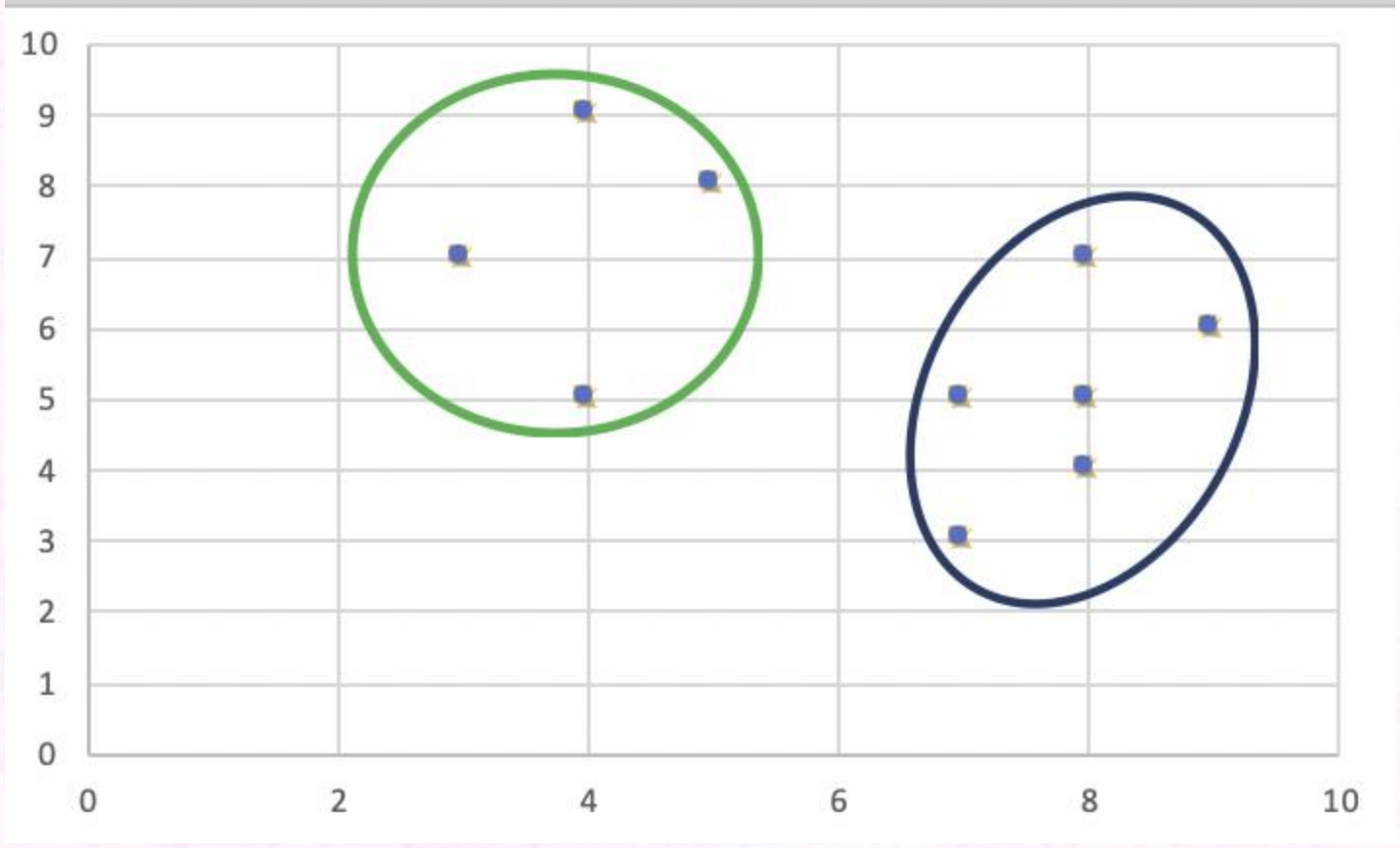
Step 2: Calculating cost. The dissimilarity of each non-medoid point with the medoids is calculated and tabulated:

	X	Y	Dissimilarity from C1	Dissimilarity from C2
0	8	7	6	2
1	3	7	3	7
2	4	9	4	8
3	9	6	6	2
4	8	5	-	-
5	5	8	4	6
6	7	3	5	3
7	8	4	5	1
8	7	5	3	1
9	4	5	-	-

Here we have used Manhattan distance formula to calculate the distance matrices between medoid and non-medoid points. That formula tell that **Distance = |X1-X2| + |Y1-Y2|.**

Each point is assigned to the cluster of that medoid whose dissimilarity is less. Points 1, 2, and 5 go to cluster C1 and 0, 3, 6, 7, 8 go to cluster C2. The Cost = (3 + 4 + 4) + (3 + 1 + 1 + 2 + 2) = 20

Step 3: randomly select one non-medoid point and recalculate the cost. Let the randomly selected point be (8, 4). The dissimilarity of each non-medoid point with the medoids - C1 (4, 5) and C2 (8, 4) is calculated and tabulated.



Advantages:

It is simple to understand and easy to implement.

K-Medoid Algorithm is fast and converges in a fixed number of steps.

PAM is less sensitive to outliers than other partitioning algorithms.

Disadvantages:

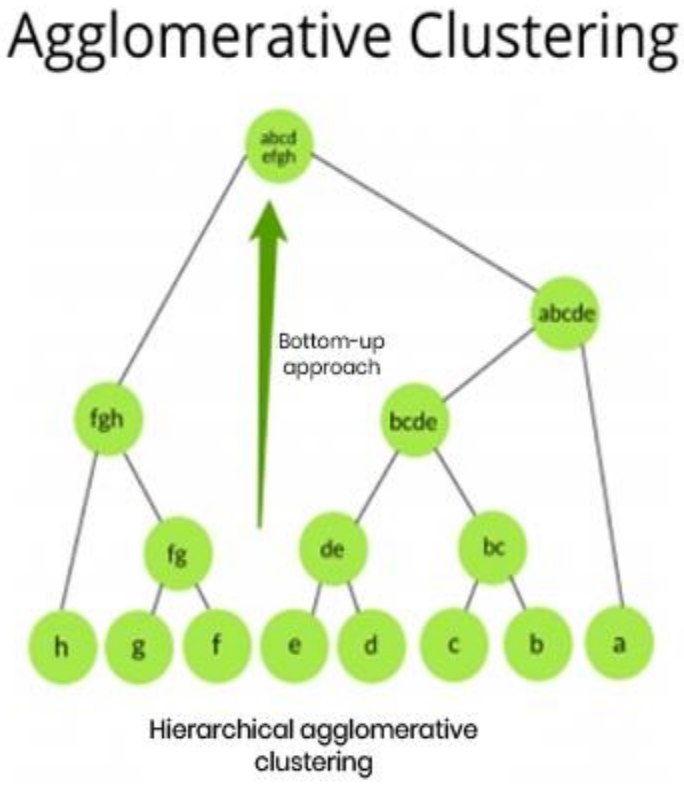
The main disadvantage of K-Medoid algorithms is that it is not suitable for clustering non-spherical (arbitrarily shaped) groups of objects. This is because it relies on minimizing the distances between the non-medoid objects and the medoid (the cluster center) - briefly, it uses compactness as clustering criteria instead of connectivity.

It may obtain different results for different runs on the same dataset because the first k medoids are chosen randomly.

Difference between K-Means and K-Medoids

K-Means	K-Medoids
Uses centroid (average of all points in cluster)	Uses medoid (an actual data point from the cluster)
Sensitive to outliers (a single extreme point can shift centroid a lot)	More robust, since medoid must be an existing point and not heavily influenced by outliers
Usually limited to Euclidean distance (because it relies on means)	Can use any distance metric (Euclidean, Manhattan, Hamming, cosine, etc.)
Faster (complexity ~ $O(n \cdot k \cdot t)$, where t = iterations)	Slower (needs pairwise distances, ~ $O(k \cdot n^2)$ per iteration)
Centroid may not be an actual point (sometimes unrealistic)	Medoid is always a real data point → more interpretable and representative

Agglomerative Hierarchical Clustering



- Agglomerative clustering is a type of hierarchical clustering that involves grouping similar objects together based on their distance. The process starts with each object being treated as its own cluster and then iteratively merging the two closest clusters until all objects belong to one cluster.
- In simple terms, imagine you have a group of people standing apart from each other. You want to form groups based on who stands close to whom. You start by pairing up the two closest people into a group and continue doing so until everyone belongs to one big group.
- Agglomerative clustering is a bottom-up approach where each object initially forms its own cluster and then gradually merges with other clusters based on some distance metric until all the objects belong to one large cluster. This process creates a hierarchy of nested clusters that a dendrogram can represent.

The basic steps involved in agglomerative clustering are:

1) Assign each object to its own initial cluster

2) Compute pairwise distances between all pairs of clusters

3) Merge the two closest clusters into one new larger cluster

4) Recompute pairwise distances between this newly merged cluster and all other remaining ones.

5) Repeat steps 3-4 until all objects belong to one large final cluster.

The basic steps involved in agglomerative clustering are:

1) Assign each object to its own initial cluster

2) Compute pairwise distances between all pairs of clusters

3) Merge the two closest clusters into one new larger cluster

4) Recompute pairwise distances between this newly merged cluster and all other remaining ones.

5) Repeat steps 3-4 until all objects belong to one large final cluster.

Distance Metrics (Linkage Criteria) in Hierarchical Clustering

When we merge clusters, we need a way to decide **how far apart two clusters are**. That's what these linkage methods do.

1. **Single Linkage (Minimum Distance)**
2. **Complete Linkage (Maximum Distance)**
3. Average Linkage
4. Centroid Linkage

Single Linkage (Minimum Distance)

Distance between two clusters = distance between the **closest points** of the two clusters.

Example:

Cluster 1 has points {A, B}

Cluster 2 has points {C, D}

Distance = min(distance(A, C), distance(A, D), distance(B, C), distance(B, D))

2. **Complete Linkage (Maximum Distance)**

Distance between two clusters = distance between the **farthest points** of the two clusters.

Example:

Distance = max(distance(A, C), distance(A, D), distance(B, C), distance(B, D))

3. **Average Linkage**

Distance between two clusters = **average distance** between all pairs of points (one from each cluster).

Example:

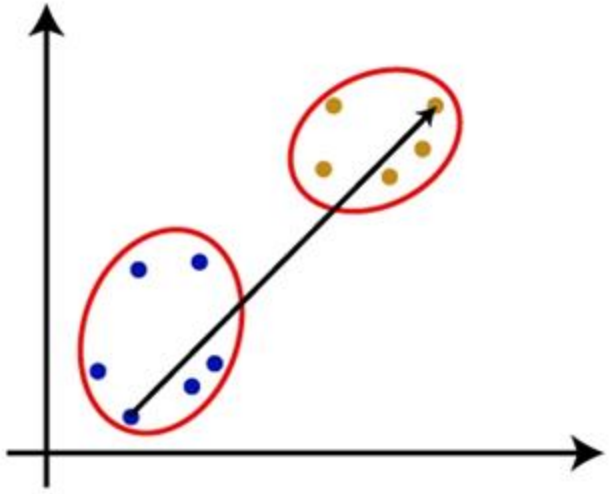
Distance = (distance(A, C) + distance(A, D) + distance(B, C) + distance(B, D)) ÷ 4

Balanced: Not too strict like complete, not too loose like single.

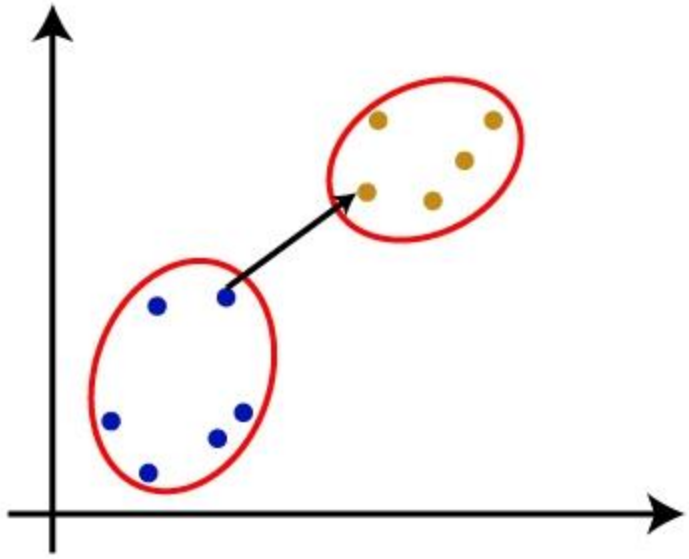
What is Agglomerative Linkage?

Agglomerative linkage refers to the method used to calculate distances between clusters during agglomerative clustering. There are several types of linkage methods, such as complete, single, and average.

Complete-Linkage: This method computes the distance between two clusters by taking the maximum pairwise distance between any two points in each cluster. It tends to produce compact and well-separated clusters but can be sensitive to outliers.



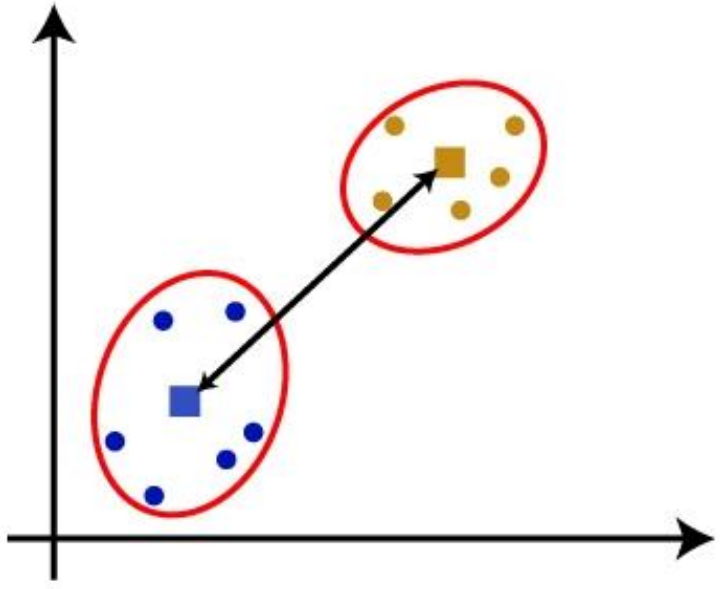
Single-Linkage: This method computes the distance between two clusters by taking the minimum pairwise distance between any two points in each cluster. It tends to produce long chains of connected objects that may not be very meaningful



Average-Linkage: This method computes the distance between two clusters by averaging all pairwise distances among their members. It balances out both extremes of complete and single linkages and produces more balanced dendrograms with moderate-length branches

Centroid Linkage: The distance between the centroids is calculated in this type of linkage

.



Agglomerative Cluster Example with Single Linkage

We have a dataset with five data points: **A, B, C, D, E,** The initial pairwise distance matrix is given below

	A	B	C	D	E
A	0	9	3	6	11
B	9	0	7	10	10
C	3	7	0	9	2
D	6	10	9	0	8
E	11	10	2	8	0

In **single linkage**, the distance between two clusters is the **shortest distance** between any single point in the first cluster and any single point in the second cluster

- **Step 1: Initialization**: Treat each data point as its own cluster {A}, {B}, {C}, {D}, {E}
- **Step 2: Find the Minimum Distance:** The minimum distance is **2**, which occurs between **C and E** (d(C, E) = 2). **Merge C and E** to form a new cluster: **{C, E}**
- **Step 3: Update the Distance Matrix:** Calculate the distance from the new cluster {C, E} to all other points (A, B, D)

- $d(\{C, E\}, A) = \min(d(C, A), d(E, A)) = \min(3, 11) = 3$
- $d(\{C, E\}, B) = \min(d(C, B), d(E, B)) = \min(7, 10) = 7$
- $d(\{C, E\}, D) = \min(d(C, D), d(E, D)) = \min(9, 8) = 8$

- Updated Matrix 1:

	A	B	{C, E}	D
A	0	9	3	6
B	9	0	7	10
{C, E}	3	7	0	8
D	6	10	8	0

- **Step 4: Second Merge:** Find the minimum distance in the updated matrix: it is **3** between **A and {C, E} then Merge A** with {C, E} to form a larger cluster: **{A, C, E}**

- Step 5: Update the Distance Matrix Again

- Compute distances from $\{A, C, E\}$ to remaining clusters (B and D):
  - $d(\{A, C, E\}, B) = \min(d(A, B), d(C, B), d(E, B)) = \min(9, 7, 10) = 7$
  - $d(\{A, C, E\}, D) = \min(d(A, D), d(C, D), d(E, D)) = \min(6, 9, 8) = 6$

- Updated Matrix 2:

	{A, C, E}	B	D
{A, C, E}	0	7	6
B	7	0	5
D	6	5	0

- Step 6: Third Merge: minimum distance: it is **5** between **B and D**. **Merge B and D** to form: **{B, D}**

- Step 7: Final Merge
- The distance between them is:d({A, C, E}, {B, D} = min(d(A,B), d(A,D), d(C,B), d(C,D), d(E,B), d(E,D)) = min(9, 6, 7, 9, 10, 8) = 6)

- **Merge {A, C, E} and {B, D}** into the single root cluster: **{A, B, C, D, E}**

Evaluating Clustering Performance

Internal Evaluation (No Ground Truth)	External Evaluation (With Ground Truth)
Silhouette Score	Adjusted Rand Index (ARI)
Davies-Bouldin Index (DBI)	Normalized Mutual Info (NMI)
Dunn Index	Purity

Clustering Metrics in Machine Learning

There are multiple clustering algorithms, and each has its way of grouping data points. Clustering metrics are used to evaluate all these algorithms. Let us take a look at some of the most commonly used clustering metrics:

1. Silhouette Score
2. Davies-Bouldin Index
3. Calinski-Harabasz Index (Variance Ratio Criterion)
4. Adjusted Rand Index (ARI)
5. Mutual Information (MI)

These clustering metrics help in evaluating the quality and performance of clustering algorithms, allowing for informed decisions when selecting the most suitable clustering solution for a given dataset.

Clustering Metrics in Machine Learning

1. The **Silhouette Score** is a way to measure **how good** the clusters are in a dataset. It helps us understand how well the data points have been **grouped**. The score ranges **from -1 to 1.**

- A score close to 1 means a point fits really well in its group (cluster) and is far from other groups.
- A score close to 0 means the point is on the border between two clusters.
- A score close to -1 means the point might be in the wrong cluster.
- **Silhouette Score (S) for a data point i is calculated as:**

$$S(i) = \frac{b(i)-a(i)}{max(a(i),b(i))}$$

where,
a(i) is the average distance from i to other data points in the same cluster.
b(i) is the smallest average distance from i to data points in a different cluster.

2. Davies-Bouldin Index

The **Davies-Bouldin Index (DBI)** helps us measure how **good the clustering** is in a dataset. It looks at how **tight each cluster** is (compactness), and **how far apart** the clusters are (separation).
Lower DBI = better, clearer clusters
Higher DBI = messy, overlapping clusters
A **lower score is better**, because it means:
Points in the same cluster are close to each other.
Different clusters are far apart from one another.

$$DB = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i}\left(\frac{R_{ii}+R_{jj}}{R_{ij}}\right)$$

where,
k is the total number of clusters.
Rii is the compactness of cluster i.
Rjj is the compactness of cluster j.
Rij is the dissimilarity (distance) between cluster i and cluster j.

Clustering Metrics in Machine Learning

3. Calinski-Harabasz Index (Variance Ratio Criterion)

The **Calinski-Harabasz Index** measures **how good** the clusters are in a dataset.

It looks at:

How **close** the points are **inside** each cluster?

How far apart the clusters are?

A **higher score is better**, as it means the clusters are tight and well-separated.

It helps determine the ideal number of clusters.

Calinski-Harabasz Index (CH) is calculated as:

$$CH = \frac{B}{W} \times \frac{N-K}{K-1}$$

where,

B is the sum of squares between clusters.

W is the sum of squares within clusters.

N is the total number of data points.

K is the number of clusters.

Calculating between group sum of squares (B)

$$B = \sum_{k=1}^{K} n_k \times ||C_k - C||^2$$

where,

nk is the number of observation in cluster 'k'

Ck is the centroid of cluster 'k'

C is the centroid of the dataset

K is number of clusters

Calculating within the group sum of squares (W)

$$W = \sum_{i=1}^{n_k} ||X_{ik} - C_k||^2$$

where,

nk*nk* is the number of observation in cluster 'k'

Xik*Xik* is the i-th observation of cluster 'k'

Ck*Ck* is the centroid of cluster 'k'

Clustering Metrics in Machine Learning

4. Adjusted Rand Index (ARI)

The **Adjusted Rand Index (ARI)** helps us measure how **accurate a clustering result** is by comparing it to the true labels (ground truth).

It checks how well the pairs of points are grouped:

- Are the same pairs together in both the real and predicted clusters?
- Are different pairs also kept apart correctly?

The score ranges from -1 to 1:

1 means perfect match - the clustering is exactly right.

0 means random guess - no better than chance.

Below 0 means worse than random - very poor clustering.

Adjusted Rand Index (ARI) is calculated as:

$$ARI = \frac{(RI - Expected_{RI})}{(max(RI) - Expected_{RI})}$$

where,

RI is the Rand Index.

ExpectedRI is the expected value of the Rand Index.

5. Mutual Information (MI)

Mutual Information measures how much two variables are related or connected. In clustering,

it **compares how much the true cluster labels match with the predicted labels**.

It shows how much knowing about one variable helps us predict the other.

The more agreement there is, the higher the score.

Higher values mean better agreement between the clusters.

Zero means no agreement at all.

MI between true labels Y and predicted labels Z is calculated as:

$$MI(y,z) = \sum_i \sum_j p(y_i, z_j) \cdot \log\left(\frac{p(y_i,z_j)}{p(y_i)\cdot p(z_j)}\right)$$

where,

yi is a true label.

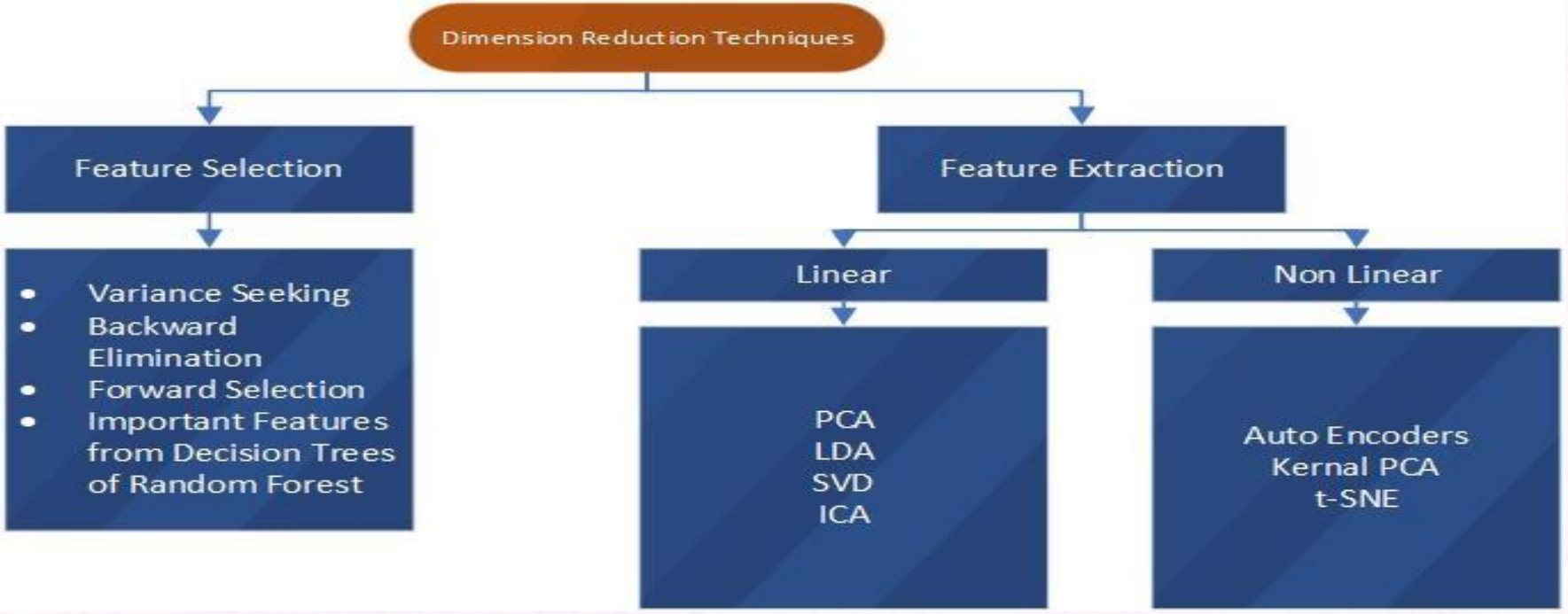
zi is a predicted label.

p(*yi*,*zi*) is the joint probability of yi*yi* and zj*zj*.

p(*yi*) and p(zi)*p*(*zi*) are the marginal probabilities.

Dimensionality Reduction using Principal Component Analysis

- **Dimensionality reduction** is a preprocessing technique in machine learning used to reduce the number of random variables or features in a dataset while retaining as much critical information as possible
- It helps combat the **"curse of dimensionality,"** which causes models to overfit, run slowly, and require massive amounts of computation and storage



Principal Component Analysis

- PCA is popular technique used to reduce the number of dimensions in large datasets while keeping the most important information
- It changes data with many variables into a smaller set of new, uncorrelated variables called **principal components**
- These new components capture the highest amount of **variance** (spread or variation) from the original data
- The first principal component holds the most variation, and each next component holds the remaining variation at a right angle (perpendicular) to the last

How PCA Works

- **Standardize**: Scale the data so every feature has an equal range
- **Covariance Matrix**: Compute the covariance matrix to see how features relate to each other
- **Eigenvalues and Eigenvectors**: Calculate eigenvectors (directions) and eigenvalues (importance) from the covariance matrix
- **Select Components**: Pick the top components with the highest eigenvalues
- **Transform**: Project the original data onto these new axes to create a smaller dataset

Pros and Cons of PCA

Pros:

- Speeds up machine learning training time.
- Removes noisy or redundant features.
- Makes high-dimensional data easy to plot and visualize

Cons:

- The new components are mixed combinations of original variables, making them hard to interpret.
- It assumes linear relationships between variables